



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Name: Kreanne C. Astillero	Date: 9-25-26
Section: BSIT 2B	Instructor: Ma'am Max



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

- I. **Objective:** Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

Answer:

This program is an IT Automation Incident Ticket Manager designed to help support teams track and manage incident tickets generated by automation bots. It stores each ticket's Incident ID, Bot name, and Short Description in a list, which serves as the program's linear data structure. Through a simple menu, the user can add new tickets, display all currently active tickets, search for a specific ticket by its Incident ID, remove a ticket once it has been resolved, and view the total count of active tickets at any time. The program was tested using the 10 sample incident tickets provided in the activity to confirm that each operation—adding, displaying, searching, removing, and counting—works correctly.

- II. **Program Operations:** Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

Add Incident: The Add Incident operation is implemented in the `add_ticket()` function. When the user provides an Incident ID, Bot name, and Short Description, the program first checks for duplicates by looping through the existing `tickets` list to see if the same Incident ID already exists. If no duplicate is found, it creates a dictionary `ticket = {"id": incident_id, "bot": bot, "desc": description}` to hold the three fields, then adds it to the collection using `tickets.append(ticket)`. The `append()` method inserts the new ticket at the end of the list, which represents adding a new active incident ticket to the system.

Display Incidents: The Display Incidents operation is implemented in the `display_tickets()` function. It performs a linear traversal of the entire `tickets` list using the loop `for t in tickets:`, going through each ticket one by one from the beginning to the end of the list. For every ticket, it prints the Incident ID, Bot, and Short Description in a neatly formatted table using `print(f"{t['id']:<15}{t['bot']:<20}{t['desc']}")`. If the list has no tickets, the program instead displays a message stating that there are no active incident tickets.



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

Search Incident: The Search Incident operation is implemented in the `search_ticket()` function using a linear search algorithm. The program scans the `tickets` list one element at a time with `for index, t in enumerate(tickets):`, comparing each ticket's ID to the Incident ID entered by the user through the condition `if t["id"].upper() == incident_id.upper():`. The comparison is done in uppercase to make the search case-insensitive. As soon as a match is found, the program displays the ticket's details along with its position in the list; if the loop finishes without finding a match, it informs the user that the ticket does not exist.

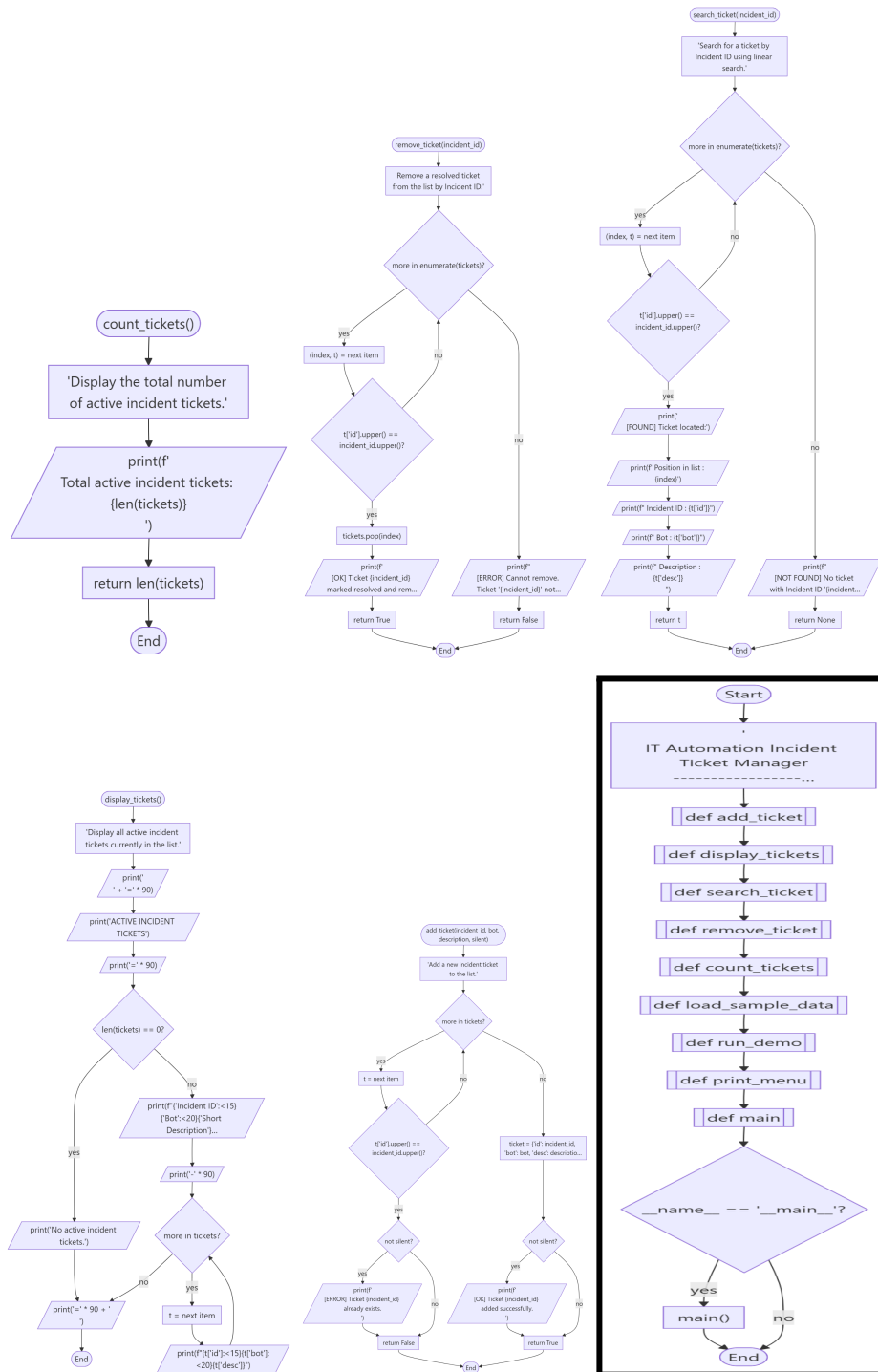
Remove Resolved Incident: The Remove Resolved Incident operation is implemented in the `remove_ticket()` function. Similar to the search operation, it scans the `tickets` list to locate the ticket whose Incident ID matches the one provided by the user. Once the matching ticket and its index are found, the program calls `tickets.pop(index)` to remove that ticket from the list. This deletes the ticket from the active list entirely, simulating that the incident has been resolved and no longer needs tracking.

Count Active Incidents: The Count Active Incidents operation is implemented in the `count_tickets()` function. It uses Python's built-in `len()` function on the `tickets` list, as shown in `print(f"Total active incident tickets: {len(tickets)}")`. Since `len(tickets)` returns the number of elements currently stored in the list, this gives the user an accurate, real-time count of how many incident tickets are still active in the system.



College of Computer Studies

III. **Flowchart:** Create a flowchart showing the overall process and logic of your program.





Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

- IV. **Conclusion:** Briefly explain what you learned from the activity and how the linear data structure was applied.

From this activity, I learned how a linear data structure, specifically a list, can be used to solve a simple but realistic IT problem, which is managing incident tickets created by automation bots. By storing each ticket's Incident ID, Bot, and Short Description as a record in a list, I understood how the basic operations we discussed in class are applied in a working program. Adding a ticket showed me how `append()` inserts a new record at the end of the list, while displaying tickets helped me understand how a loop can go through the whole list one by one to show all the data. Searching and removing a ticket taught me how linear search works, where the program checks each item in the list until it finds a match, and how that same idea is used to find and delete a specific record. Counting the active tickets also showed me that the `len()` function can easily tell how many items are currently stored in the list. Overall, this activity helped me understand more about how linear data structures work in an actual program, and not just in theory or diagrams. It made me realize that even a simple list can be very useful in organizing and managing real data, which is something I can apply in our future IT-related projects.